on is to enable you to understand which sources are at your disposal and which ones you may wish to select for a certain task.

In this chapter, we will categorize sources using a simple and widely used trichotomy: compute, storage, and network. On a high level, this looks as follows:

- *Compute*—Runtime environments, such as a virtual machine (VM), including those that are general purpose and language specific, like the Java VM, as well as the compute unit, like a microservice running in a container or a function supported by AWS Lambda
- *Storage*—Relational databases (PostgreSQL, MySQL, etc.), NoSQL data stores (Redis, MongoDB, DynamoDB, etc.), file systems (local and networked, like NFS), as well as object stores (e.g., S3)
- *Network*—Network interfaces; gateways; load balancers; virtual private networks (VPNs); and entire isolated cloud provider networks, such as a virtual private cloud (VPC)

In this chapter, we will cover a wide array of technologies. The goal is not to cover each of the signal types per source in detail but to provide you with an overview and a general idea of what to expect as well as information about where to learn more about each source.

3.1 Selecting sources

Remember that the goal of observability is to gain actionable insights, which starts with knowing what sources and signals to consider. On the upside, each source we select contributes signals we can directly use or correlate with other signals. On the other hand, each source that we take into account requires investments: from the instrument costs to agents required to ingest the signals to storage costs in the destinations.

One important thing to consider in the context of sources is what source is relevant to a task or role at hand. An efficient strategy for selecting signal sources is to work backwards, on a per-task or use-case basis. For example, the task at hand might be on-call for infrastructure operations, and you might primarily care about the overall CPU, RAM, and file system utilization. Another example might be improving the latency of an API request as a developer, where you require visibility into network-related sources.

Not only must you consider which sources you select but also which signals from a source you use and to what extent. Consider, for example, the case of a serverless Kubernetes offering, where the control plane and the data plane are both managed for you. Take etcd, where the Kubernetes control plane keeps its state in this context: given that you don't have access to it, you can't change anything in the setup (e.g., rebalance or scale etcd). Because of this immutability, detailed metrics are not as valuable, whereas an overall health indicator is.

However, there are tradeoffs you need to consider; for example, while you might not control how autoscaling happens, having good visibility into how often or when it